

Heroes in Super-Training

Progettazione e validazione di un sistema ibrido basato sulla conoscenza per la classificazione di eroi multiverso.

Anno Accademico 2025/2026

Gruppo di lavoro: Michela Giulia Cericola, 778929, m.cericola@studenti.uniba.it
[Repository del progetto](#)

Indice

1	Introduzione	3
2	Sommario	3
3	Argomenti di Interesse	3
4	Dataset	3
4.1	Sommario	3
4.2	Strumenti Utilizzati	4
4.3	Decisioni di Progetto	4
4.4	Valutazione	4
5	Apprendimento Supervisionato	5
5.1	Sommario	5
5.2	Strumenti Utilizzati	5
5.3	Decisioni di Progetto	5
5.4	Valutazione	6
6	Ontologia	9
6.1	Sommario	9
6.2	Strumenti Utilizzati	9
6.3	Decisioni di Progetto	9
6.4	Valutazione	12
7	Sistema di Supporto alle Decisioni	13
7.1	Sommario	13
7.2	Strumenti Utilizzati	13
7.3	Decisioni di Progetto	14
7.4	Valutazione	15
8	Conclusioni e Sviluppi futuri	16
9	Bibliografia	18

1 Introduzione

Heroes in Super-Training è un progetto finalizzato a definire i ruoli degli eroi del multiverso DC e Marvel. La scelta di questo caso di studio è nata dal desiderio di mettermi alla prova con un dominio narrativo stimolante, scoprendo come la logica matematica potesse fare ordine in un immaginario così vasto.

2 Sommario

Il programma ha come obiettivo di sviluppare e testare comparativamente un'architettura ibrida in grado di coniugare l'apprendimento statistico induttivo con il rigore della logica simbolica e del ragionamento automatico.

Lo scenario d'analisi del progetto è focalizzato sul Multiverso dei supereroi, con particolare riferimento ai personaggi appartenenti ai due principali universi: DC Comics e Marvel.

Il sistema mira a superare i limiti dei modelli di Machine Learning, attraverso l'integrazione della Background Knowledge proveniente dal Web Semantico. Il sistema inoltre impiega la conoscenza strutturata dell'ontologia per alimentare un'interfaccia interattiva in cui l'utente può sottoporre il sistema a scenari operativi e sfide tattiche da superare.

3 Argomenti di Interesse

Gli argomenti di riferimento del corso su cui si articola il progetto sono:

- **Apprendimento Supervisionato (Machine Learning):** l'approccio data-driven è incentrato sull'addestramento di un albero di decisione per la classificazione statistica dei ruoli dei personaggi in base alle feature numeriche.
- **Rappresentazione della Conoscenza (Ontologia OWL) e Ragionamento Deduttivo:** l'ontologia modella formalmente la conoscenza di fondo del multiverso, mentre il ragionatore deduttivo (HermiT) applica le regole logiche per convalidare e correggere le risposte del Machine Learning.
- **Sistema di Supporto alle Decisioni (DSS):** modulo finale di interazione in cui il dominio viene convertito in una rappresentazione proposizionale. Il sistema permette all'utente di configurare scenari operativi e sfide tattiche, valutando e ordinando i personaggi tramite una funzione euristica per trovare la soluzione ottimale al problema proposto.

4 Dataset

4.1 Sommario

Il primo e fondamentale passo per la realizzazione del progetto Heroes in Super-Training è stato lo sviluppo del dataset da zero. Una ricerca preliminare sul web e sui repository pubblici (es. Kaggle) ha evidenziato l'assenza di basi di dati strutturate con attributi idonei a supportare l'idea di base del progetto.

La maggior parte dei dataset disponibili, infatti, presentava metadati prettamente biografici o narrativi destrutturati, privi di una conformazione matematica adatta a un compito di classificazione statistica supervisionata. Per addestrare un modello predittivo capace di associare un supereroe al suo ruolo tattico, era invece necessaria una matrice di dati precisa, in cui ogni individuo fosse descritto da caratteristiche quantitative confrontabili.

4.2 Strumenti Utilizzati

Il dataset è stato realizzato tramite sistema di gestione di database relazionali open-source (MySQL). Consecutivamente sono stati esportati i dati nel file csv presente nel progetto.

4.3 Decisioni di Progetto

È stata creata la tabella sottostante:

```
CREATE TABLE Super_Heroes(  
  id int PRIMARY KEY,  
  name VARCHAR(25) NOT NULL,  
  universe VARCHAR(2) CHECK (universe IN ('DC', 'MC')),  
  strength INT CHECK (strength BETWEEN 1 AND 10) NOT NULL,  
  intelligence INT CHECK (intelligence BETWEEN 1 AND 10) NOT NULL,  
  speed INT CHECK (speed BETWEEN 1 AND 10) NOT NULL,  
  role VARCHAR(10) CHECK (role IN ('Leader', 'Powerhouse', 'Specialist')) NOT NULL,  
  power_source VARCHAR(25) CHECK (power_source IN ('Supernatural', 'Technological_Weapon')) NOT NULL,  
  popularity INT CHECK (popularity BETWEEN 1 AND 10) NOT NULL  
);
```

Nella quale ho optato per la scelta di tali attributi:

Id - identificativo univoco del super eroe;

name - nome dell'eroe, valore variabile con soglia 25 caratteri;

universe - universo dell'eroe, scelta tra due mondi (DC e MC). Utile nella terza fase del progetto in cui vediamo confrontarsi i due mondi;

strenght - forza dell'eroe, intero vincolato da 1 a 10;

intelligence - intelligenza dell'eroe, intero vincolato da 1 a 10;

speed - velocità dell'eroe, intero vincolato da 1 a 10;

role - ruolo dell'eroe, valore tra Leader, Powrhouse (inteso come potenza), Specialist. Il ruolo è stato posto all'interno della tabella per verificare la veridicità e l'attendibilità, più avanti verrà approfondito;

power source - provenienza del potere dell'eroe, scelta tra soprannaturale e arma tecnologia;

popularity - popolarità dell'eroe, intero vincolato da 1 a 10;

La conoscenza personale di tale argomento e la ricerca sul web di alcuni personaggi che non mi erano noti, sono stati di supporto per la popolazione della tabella e dei valori in essa stessa.

4.4 Valutazione

Gli attributi selezionati in questa fase si sono rivelati altamente decisivi e dotati di un'elevata attendibilità predittiva.

5 Apprendimento Supervisionato

5.1 Sommario

L'apprendimento supervisionato crea un modello in grado di prevedere output corretti su dati mai visti prima, estraendo dai dati grezzi un insieme di regole logiche.

Questo primo blocco di funzionamento del programma concerne nella previsione dei ruoli degli eroi. I ruoli in questione sono: Leader, Powerhouse, Specialist.

5.2 Strumenti Utilizzati

Il linguaggio a cui fa riferimento la creazione di questo progetto è Python. Questa scelta è legata alla sua versatilità e alla disponibilità di un ricco ecosistema di librerie. Infatti, per questo modulo sono state utilizzate:

- **pandas (pd):** gestisce il dataset caricando il file CSV e manipolando i dati tramite strutture tabellari;
- **numpy (np):** esegue calcoli matematici rapidi su vettori, come il calcolo dell'accuratezza media;
- **scikit-learn (sklearn)** utilizza:
 - **DecisionTreeClassifier:** l'algoritmo che addestra l'albero di decisione per classificare i ruoli degli eroi;
 - **export_text:** estrae e mostra sotto forma di testo le regole logiche applicate dall'albero;
 - **accuracy_score:** calcola la percentuale globale di risposte corrette del modello;
 - **classification_report:** genera il resoconto dettagliato con le metriche di Precision, Recall e F1-Score;
 - **cross_val_score:** esegue i test ripetuti sulle diverse combinazioni di dati;
 - **KFold:** divide i 130 eroi in 5 blocchi da 26 personaggi per la validazione incrociata;
- **matplotlib.pyplot (plt):** genera e mostra i grafici visivi per analizzare l'andamento e i risultati delle simulazioni.

5.3 Decisioni di Progetto

In questa fase del progetto, si procede inizialmente con l'importazione del dataset tramite la lettura del file CSV precedentemente generato. Successivamente, viene eseguita un'operazione di preprocessing dei dati, applicando una codifica per convertire l'attributo qualitativo **power source** in valore numerico, rendendolo così elaborabile dall'algoritmo di classificazione.

Gli attributi selezionati sono: *strength*, *intelligence*, *speed*, *power_source*, *popularity*.

Queste caratteristiche giocano un ruolo fondamentale nel progetto, poiché permettono di definire metriche quantitative e qualitative precise, necessarie all'algoritmo per discriminare e comprendere al meglio il ruolo di ciascun eroe (*Leader*, *Powerhouse*, *Specialist*).

Per la predizione del ruolo, infatti, ci serviamo del classificatore **Decision Tree**. L'apprendimento del nostro modello esegue una ricerca euristica per identificare i punti di scissione ottimali. Questo processo viene applicato in modo ricorsivo top-down (dall'alto verso il basso): partendo

dal dataset completo di 130 eroi nel nodo radice, l'algoritmo seleziona l'attributo più discriminante in base al criterio di Gini per effettuare il primo taglio.

Perchè la scelta del decision tree?

Il *Decision Tree Classifier* è l'algoritmo cardine per questo modulo di apprendimento supervisionato. Nella moderna Ingegneria dei Dati, l'adozione di modelli complessi o di tipo *ensemble*, come Random Forest o le Reti Neurali, garantisce spesso prestazioni statistiche di alto livello; tuttavia, tali architetture operano intrinsecamente come scatole nere (*Black-Box*), rendendo matematicamente impossibile l'ispezione dei singoli passaggi logici che conducono alla classificazione.

```
decision_tree = DecisionTreeClassifier(  
    max_depth=3,  
    criterion='gini',  
    random_state=10  
)
```

Figura 1: Configurazione degli iperparametri del DecisionTreeClassifier.

Al contrario, l'Albero di Decisione si configura come un modello trasparente (*White-Box*). Questa caratteristica si rivela fondamentale per l'interoperabilità con il secondo blocco del programma, incentrato sul Web Semantico. L'algoritmo, infatti, mappa lo spazio delle *feature* geometriche che nel codice viene arrestata alla profondità massima di tre livelli (`max_depth=3`).

Un ulteriore elemento a supporto di questa scelta risiede nell'utilizzo del criterio Gini (`criterion='gini'`). L'indice di Gini ottimizza i tempi di addestramento misurando la pura probabilità di una classificazione errata di un elemento scelto casualmente. Questo approccio matematico si adatta con la gestione di un dataset ibrido, composto sia da parametri fisici (*strength*, *intelligence*, *speed*, *popularity*) sia da metadati qualitativi codificati (*power_source*), senza risentire in alcun modo delle differenti scale numeriche degli attributi.

Successivamente per la valutazione dell'affidabilità di tale modello si è optato per una tecnica di ricampionamento: *k-fold Cross Validation*, in questo caso particolare *k* equivale a 5. Divide il dataset in 5 parti uguali, scelte casualmente, ad ogni iterazione, 4 fold vengono usati per l'addestramento (*training*) e il fold rimanente per la validazione (*validation*).

Infine, l'analisi quantitativa è supportata da uno studio grafico mediante la libreria `matplotlib`.

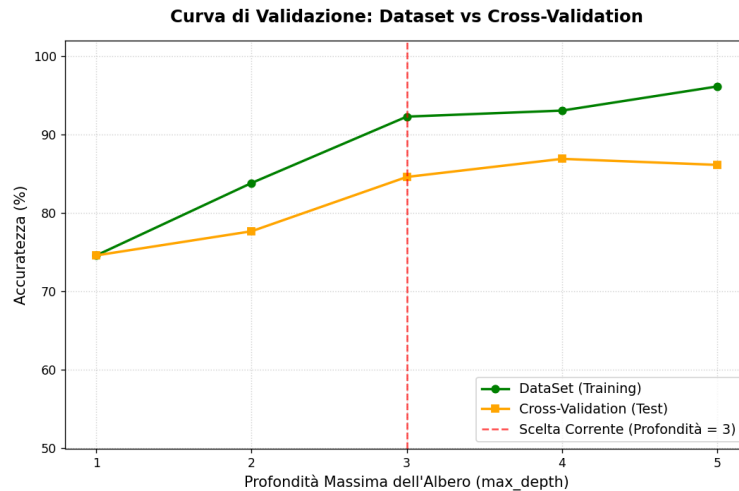
5.4 Valutazione

In fase di valutazione rispetto questa parte del progetto si evince: la scelta più ottimale per l'ampiezza dell'albero di decisione è 3.

Lo possiamo notare dalla visualizzazione del grafico sottostante:

Il grafico mostra l'andamento dell'accuratezza del modello al variare della profondità massima dell'albero (`max_depth`), confrontando le prestazioni sul set di addestramento (*Training*, linea verde) con quelle ottenute tramite la validazione incrociata (*Cross-Validation*, linea arancione).

L'analisi del diagramma evidenzia tre comportamenti fondamentali:



- **Underfitting per $\text{max_depth} \leq 2$:** con profondità ridotta, l'albero non ha la capacità strutturale di evidenziare la complessità dei dati. Le regole estratte sono troppo generiche per discriminare correttamente i tre ruoli tattici.
- **Overfitting per $\text{max_depth} \geq 4$:** aumentando la profondità, si nota una netta divergenza tra le due linee. La curva di *Training* continua a salire quasi linearmente, mentre la curva di *Cross-Validation* si stabilizza e presenta un accenno negativo a profondità 5. Vi è una perdita di generalizzazione.
- **Punto ottimale e trade-off ($\text{max_depth} = 3$):** identificato dalla linea tratteggiata rossa, profondità 3 rappresenta il perfetto punto di equilibrio del sistema. In questa configurazione, l'albero raggiunge un'ottima accuratezza di validazione (circa 85%) mantenendo uno scarto ridotto rispetto alla curva di addestramento.

Questa evidenza convalida la scelta progettuale dell'iperparametro pari a 3: il modello apprende le regole generali del dominio senza specializzarsi eccessivamente.

Per una valutazione approfondita dell'algoritmo di classificazione, l'analisi non si focalizza unicamente sull'accuratezza globale, ma monitora quattro metriche fondamentali estratte dal `classification_report` al termine del processo:

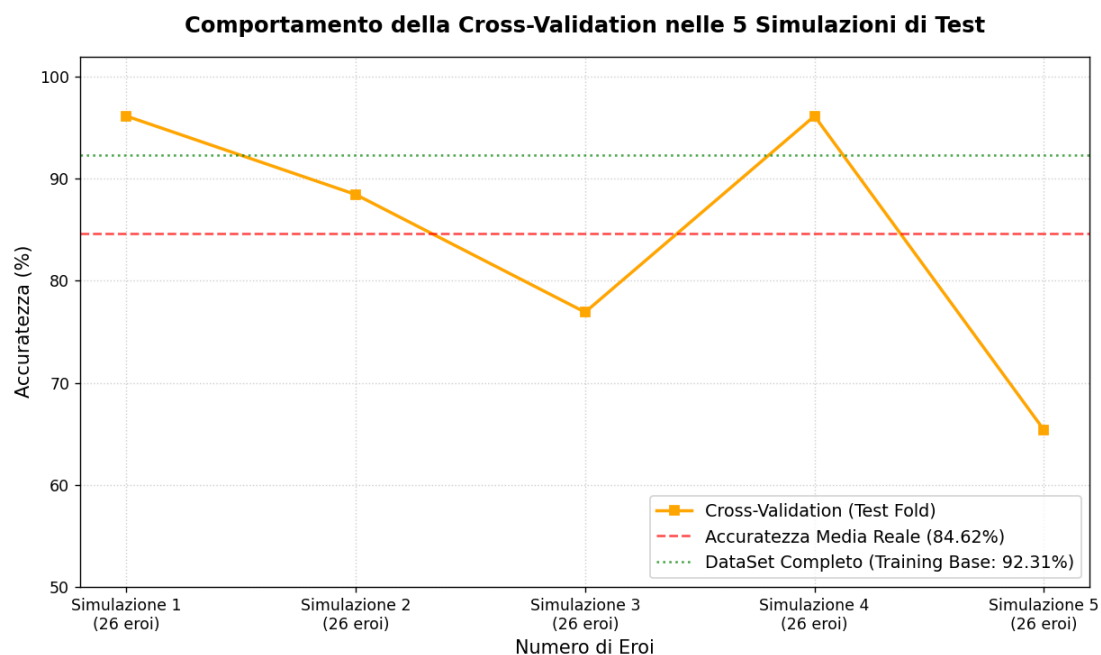
- **Accuracy:** la percentuale globale di ruoli correttamente predetti sul totale dei casi analizzati.
- **Precision Media:** la capacità del modello di non classificare come appartenenti a un ruolo (es. *Leader*) eroi che in realtà non lo sono, riduce l'incidenza dei falsi positivi.
- **Recall Medio:** la capacità dell'albero di intercettare e identificare tutti gli eroi appartenenti a una determinata classe, riduce i falsi negativi.
- **F1-Score Medio:** verifica la stabilità del modello su classi potenzialmente sbilanciate.

CLASSIFICAZIONE				
	precision	recall	f1-score	support
Leader	1.00	0.84	0.91	25
Powerhouse	0.86	0.95	0.90	39
Specialist	0.94	0.94	0.94	66
accuracy			0.92	130
macro avg	0.93	0.91	0.92	130
weighted avg	0.93	0.92	0.92	130

Una prima valutazione sul set di addestramento ha mostrato un'accuratezza del 92% circa, dato però non significativo poichè calcolato su dati già visti dal modello. La valutazione affidabile è quella ottenuta dalla Cross-validation.

Verifichiamo se con la Cross-Validation questi parametri prestazionali mantengono stabilità. Sottoponendo l'albero alle varie simulazioni di test indipendenti previsti dal protocollo di validazione.

E' possibile visionare il grafico della Cross-Validation sulle 5 simulazioni:



L'aspetto più rilevante dell'esperimento risiede nella *Deviazione Standard*, pari a $\pm 11.92\%^{**}$. Questa marcata oscillazione delle performance tra le singole simulazioni, che spaziano dai picchi d'eccellenza stabili delle Simulazioni 1 e 4 (96.15%) fino al crollo critico registrato nella Simulazione 5 (65.38%), evidenzia una **forte sensibilità** dell'albero alla combinazione dei dati estratti.

```
RISULTATI CROSS-VALIDATION
Simulazione 1 (su 26 eroi): 96.15%
Simulazione 2 (su 26 eroi): 88.46%
Simulazione 3 (su 26 eroi): 76.92%
Simulazione 4 (su 26 eroi): 96.15%
Simulazione 5 (su 26 eroi): 65.38%

Accuratezza MEDIA Reale: 84.62%
Deviazione Standard : +/- 11.92%
```

Nella prossima fase, grazie all'ausilio della componente semantica e del ragionatore deduttivo, controlleremo come sia possibile intervenire a posteriori su queste forti fluttuazioni prestazionali.

6 Ontologia

6.1 Sommario

L'esigenza di strutturare il sapere relativo all'universo dei supereroi ha spinto verso l'adozione di un approccio ontologico, ideale per mappare in modo formale la gerarchia delle classi, i legami semantici e le singole entità degli universi Marvel e DC.

Scritta in linguaggio OWL, l'infrastruttura si divide nettamente tra la *TBox* (definisce vincoli e regole dei profili tattici), e la *ABox* (accoglie i dati e le caratteristiche specifiche di ogni personaggio).

Questa base di conoscenza coopera con l'algoritmo predittivo, rettificandone le anomalie statistiche.

6.2 Strumenti Utilizzati

Questa parte di progetto utilizza un editor per la creazione di ontologia: [Protégé](#). Dopo la creazione viene esportata l'ontologia in formato OWL.

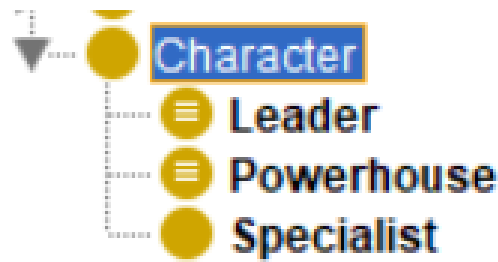
Ci serviamo della libreria: `owlready2`. Essa permette di caricare ontologie OWL 2.0 come oggetti Python, modificarle, salvarle ed eseguire ragionamenti tramite architettura HermiT.

6.3 Decisioni di Progetto

La forte specificità del dominio d'indagine ha fatto sì che si optasse per una creazione da 0 dell'ontologia.

La classe cardine del dominio è rappresentata da **Character**, da cui si diramano i diversi ruoli tattici come sottoclassi: **Leader**, **Powerhouse** e **Specialist**.

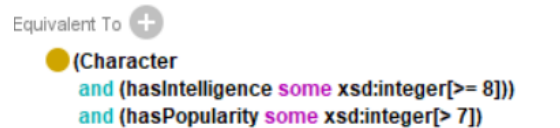
È fondamentale notare come **Leader** e **Powerhouse** non siano semplici sottoclassi primitive, ma



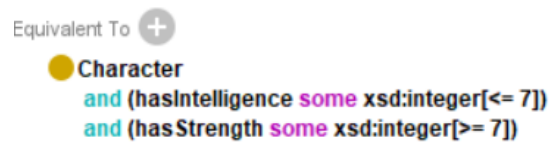
classi definite mediante condizioni equivalenti (contraddistinte dal simbolo $\equiv$).

Ciò significa che possiedono assiomi formali di classificazione: qualsiasi individuo inserito nella ABox che rispetti determinati criteri parametrici verrà automaticamente riconosciuto dal *Reasoner* (Hermit) come membro di quella specifica classe, abilitando il ragionamento deduttivo automatico senza interventi manuali. L'integrazione di parametri numerici (`xsd:integer`) e restrizioni sui tipi di dati nei costruttori di classe risponde a precise necessità ingegneristiche e architetturali del progetto, definendo la semantica formale dei ruoli:

- **Definizione Parametrica del Ruolo Leader:** un'entità viene riconosciuta come equivalente a un **Character** se e solo se possiede contemporaneamente un livello di intelligenza molto alto ed elevati valori di carisma o popolarità.



- **Definizione Parametrica del Ruolo Powerhouse:** è definito come un **Character** che, pur non spiccando necessariamente per perspicacia, compensa con una forza fisica devastante sul campo.



- **Definizione del Ruolo Specialist:** la classe è stata lasciata intenzionalmente priva di vincoli parametrici stringenti perché i personaggi che vi ricadono non mostrano un pattern di caratteristiche numeriche dominanti o rilevanti, l'approccio formale corretto in OWL è descriverla come una classe primitiva.

Perché la scelta di parametri numerici? E' importante notare che le soglie numeriche utilizzate per Leader e Powerhouse coincidono con attributi già presenti nel dataset e usati dal classificatore statistico.

Questo non rende la TBox una semplice riformulazione del target: il valore aggiunto del ragionamento deduttivo non risiede nel singolo controllo di soglia (di per sé computazionalmente banale), ma nell'effetto sistemico dimostrato nella fase di valutazione.

Il sistema è capace di correggere errori statistici del Decision Tree, tramite vincoli logici indipendentemente dai fold in fase training. Questa correzione si attiva sia attraverso il meccanismo di classificazione per fallback sui modelli Specialist sia mediante l'applicazione di vincoli di disgiunzione tra le classi tutte dinamiche che richiedono comunque una verifica finale di consistenza da parte del reasoner.

Il modulo predittivo ragiona in modo statistico-probabilistico ed è soggetto a fallire davanti a eccezioni narrative o anomalie di campionamento. Inserendo queste regole logiche nette, l'ontologia impone una verità assiomatica deterministica. Se il *Decision Tree* sbaglia la classificazione di un eroe anomalo, l'ontologia interviene a posteriori per sovrascrivere l'errore statistico basandosi su questi sbarramenti matematici oggettivi.

Per gestire la comunicazione tra la pipeline algoritmica e l'infrastruttura semantica è stata impiegata la libreria `owlready2` in ambiente Python, la quale offre le API necessarie per manipolare programmaticamente classi, proprietà e individui del grafo logico. L'architettura del modulo `semantic_hero` si articola su due routine principali volte ad automatizzare il flusso dei dati e l'estrazione di nuova conoscenza:

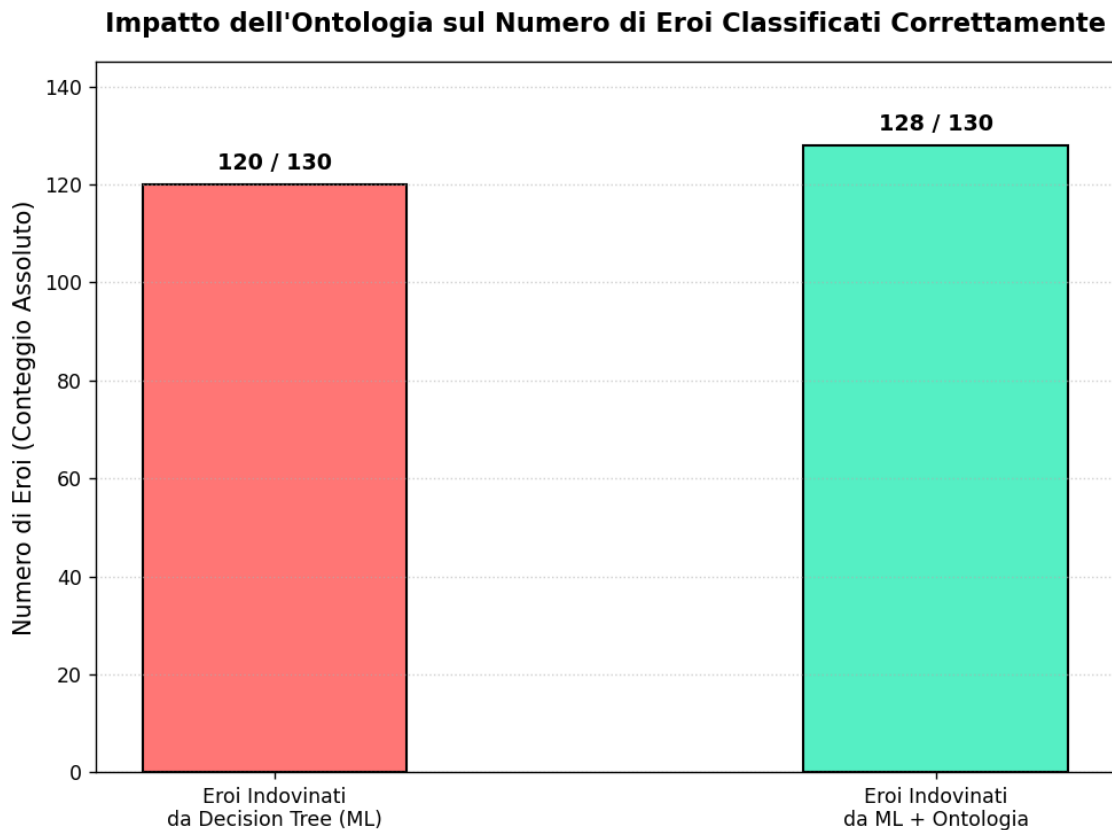
- `populate_ontology()`: la funzione carica la struttura assiomatica (**TBox**) dal file `super_heroes_empty.owl` e, scorrendo il dataset tramite libreria `pandas`, genera dinamicamente nella **ABox** gli individui della classe **Character**. Per ciascun eroe il nome viene normalizzato sostituendo gli spazi con caratteri *underscore* e vengono associate le relative *Data Properties* quantitative (`hasStrength`, `hasIntelligence`, `hasSpeed` e `hasPopularity`). Infine, l'ontologia popolata viene salvata nel file `super_heroes_populated.owl`.
- `run_reasoning()`: la funzione esegue il processo di inferenza deduttiva sulla **ABox** invocando il motore HermiT tramite il metodo nativo `sync_reasoner_hermit(infer_property_values=True)`. Il ragionatore analizza i vincoli della **TBox** e riclassifica automaticamente i personaggi intercettandone la gerarchia inferita (`is_a`). Qualora un eroe non venga riconosciuto come **Powerhouse** o **Leader**, il sistema applica un meccanismo di *fallback* semantico, assegnandolo alla classe primitiva **Specialist**.

Al termine della computazione logica, i verdeti semantici vengono formattati in una lista di dizionari Python contenente l'associazione tra il nome dell'eroe e il ruolo dedotto (`ruolo_ontologia`). Questo output viene restituito direttamente al modulo `main` del programma.

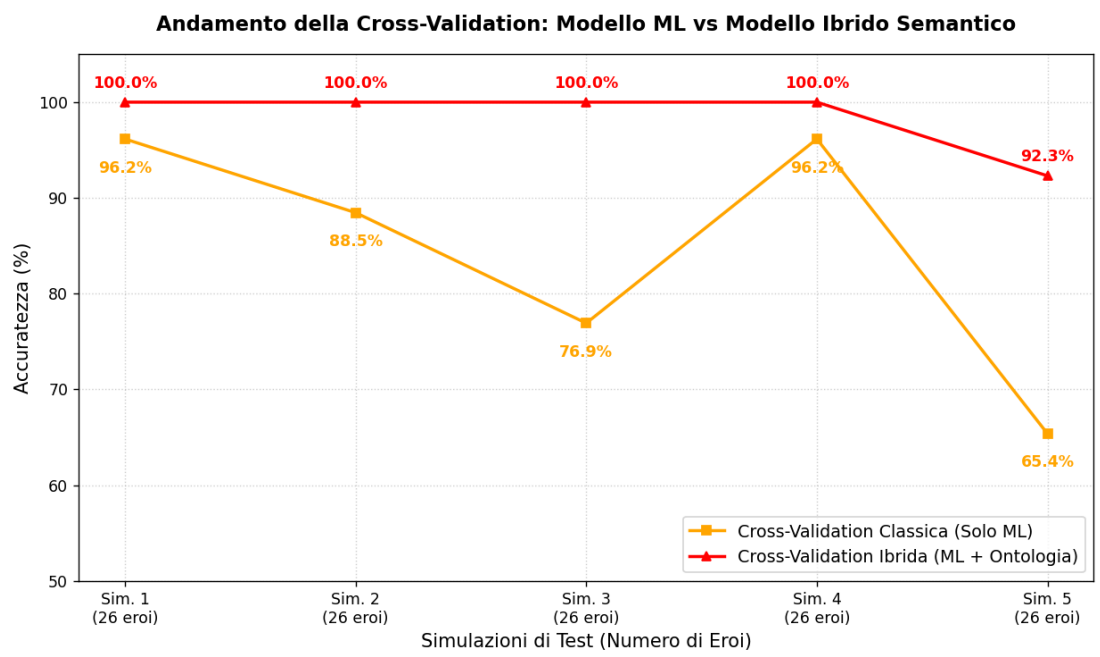
6.4 Valutazione

La base di conoscenza ontologica apporta un valore aggiunto all'interno dell'ecosistema ibrido. Il suo impatto effettivo viene dettagliato e dimostrato nella sezione conclusiva del lavoro, in cui si evidenzia come l'integrazione del ruolo tattico inferito dal ragionatore determini un drastico incremento nelle performance predittive e nella stabilità complessiva dei modelli di Machine Learning.

Infatti possiamo visualizzare dei grafici che attestino tale incremento:



Anche se a prima vista l'aumento degli eroi indovinati può sembrare ridotto (si passa da 120 a 128 eroi indovinati, su un totale di 130), la scomposizione della metrica sui singoli *fold* rivela un miglioramento strutturale legato alla **robustezza** e alla **stabilità** del sistema.



L'andamento del Modello Ibrido Semantico (rappresentato dalla curva rossa) dimostra l'efficacia dell'azione coordinata tra apprendimento statistico e vincoli logici:

Il contributo dell'ontologia non consiste nella duplicazione delle informazioni già presenti nel dataset, ma agisce come meccanismo di validazione esterno e deterministico. Esso è capace di intervenire selettivamente sui casi in cui il modello statistico fallisce, come dimostrato. Inoltre, l'obiettivo risiede nella capacità di **stabilizzare il sistema contro le anomalie dei dati**, eliminando i picchi negativi di errore.

7 Sistema di Supporto alle Decisioni

7.1 Sommario

L'ultima parte del progetto riguarda l'interazione da parte dell'utente con il sistema attraverso un menu interattivo da terminale, simulando gli scenari tattici, quali:

1. **Missione Segreta (Infiltrazione spionistica);**
2. **Simulatore di Battaglia (Marvel vs DC);**
3. **Leader Ideale (Attacco Cyber).**

Questa fase dimostra l'efficacia pratica del progetto, trasformando i dati grezzi del dataset e le inferenze astratte dell'ontologia in uno strumento decisionale concreto, trasparente ed esplicativo per la gestione della *Battle*.

7.2 Strumenti Utilizzati

L'infrastruttura matematica e di calcolo geometrico del Decision Support System fa affidamento a tre librerie di Python:

- `import numpy as np`: viene utilizzata per l'applicazione della funzione `np.argsort()`. Questo metodo permette di ordinare gli indici degli elementi in base ai punteggi di somiglianza calcolati, consentendo di estrarre e classificare i supereroi dal più idoneo al meno idoneo rispetto alla query.
- `from sklearn.feature_extraction.text import TfidfVectorizer`: analizza le stringhe semantiche che descrivono i personaggi (ruolo ontologico, universo e tratti) e le proietta in uno spazio vettoriale continuo, pesando l'importanza di ciascun attributo in base alla sua rarità e frequenza nel dataset.
- `from sklearn.metrics.pairwise import cosine_similarity`: calcola la similarità del coseno tra il vettore della query (lo scenario di battaglia) e i vettori dei supereroi nello spazio delle feature. Misurando il coseno dell'angolo compreso tra i vettori, il sistema quantifica matematicamente l'affinità strategica di un eroe, dove un valore vicino a 1 (100%) indica la massima coerenza tattica.

7.3 Decisioni di Progetto

La funzione `decision_system()` rappresenta la componente strategica dell'architettura software, integrando i verdetti deduttivi estratti dall'ontologia OWL con strumenti di recupero dell'informazione geometrico-lineare.

Poiché il costrutto dell'universo di appartenenza è mappato nel file sorgente sotto diverse chiavi nominali (es. `universe`, `faction`), il codice esegue un controllo sequenziale uniformando le stringhe nei due macro-ambiti dominanti *Marvel* e *DC*.

Il sistema recupera la proprietà `ruolo_ontologia`, ovvero il verdetto logico calcolato precedentemente dal ragionatore *HermiT*, inserendolo come elemento centrale dell'identità del personaggio.

Tramite la funzione di supporto `support()`, ogni proprietà quantitativa (*popularity*, *speed*, *strength*, *intelligence*, *power_source*) viene protetta da eventuali valori NaN o nulli, assegnando un valore di *default* ed evitando crash a run-time.

Per la struttura del profilo semantico di ogni eroe si è optato per la discretizzazione delle statistiche in tratti logici descrittivi.

```
Character_[Nome] is_classified_as_[Ruolo] belongs_to_[Universo] [Tratti]
```

Tale approccio uniforma lo spazio dei dati, allineando formalmente il linguaggio del dataset con la semantica assiomatica definita nella TBox dell'ontologia.

Vettorizzazione TF-IDF e Costruzione dello Spazio delle Feature- L'uso del TF-IDF permette di pesare l'importanza dei singoli tratti: identificatori unici o rari nel dataset assumono un peso geometrico superiore rispetto a termini generici, affinando l'accuratezza del successivo calcolo di *matching*.

Modellazione e Soluzione degli Scenari Tattici tramite Similarità del Coseno- Il sistema non si limita a filtrare i dati in modo booleano, ma affronta la complessità degli scenari di combattimento modellandoli come stringhe.

La decisione di adottare la `cosine_similarity` consente di calcolare la vicinanza angolare tra il vettore della query e i vettori dei supereroi.

Il DSS è così in grado di formulare una risposta flessibile, calcolando un punteggio di affinità percentuale e ordinando i candidati ottimali mediante la funzione `np.argsort()`.

7.4 Valutazione

Quest'ultima fase si presenta come un gioco interattivo, dove l'utente può selezionare tre possibili scenari di combattimento. Tramite un menu da terminale, l'utente sceglie la simulazione desiderata e il sistema risponde mostrando in tempo reale i supereroi più adatti, ordinati con una percentuale di affinità strategica.

```
Scegli lo scenario!  
[1] MISSIONE SEGRETA (Infiltrazione spionistica)  
    -> Seleziona eroi Specialist a basso profilo e alta mobilità.  
[2] SIMULATORE DI BATTAGLIA: SQUADRA MARVEL vs MINACCIA DC  
    -> Genera la coalizione ottimale Marvel (Powerhouse e Leader tattici) per contrastare la DC.  
[3] LEADER IDEALE (Attacco Cyber)  
    -> Chi rispecchia maggiormente il ruolo di leader per fronteggiare un attacco cyber?  
[0] Esci dal programma
```

Verifichiamo ad esempio lo scenario 1, come si comporta il sistema
 La Figura mostra l'output generato dal terminale a seguito della selezione dello Scenario 1,

```
Inserisci il numero dello scenario desiderato: 1

-----
RISPOSTA ALLA QUERY SEMANTICA: MISSIONE SEGRETA (Infiltrazione spionistica)
Target: 'is_classified_as_Specialist has_trait_low_profile has_trait_high_mobility'
-----
>> INFILTRAZIONE: Rilevato scenario ostile ad alto rischio visibilità.
>> ALGORITMO: Estrazione agenti furtivi...

Membro Suggestito      | Classe OWL      | Universo  | Affinità
-----
-> Falcon               | Specialist      | Marvel    | 41.02%
-> Beast                | Specialist      | Marvel    | 39.48%
-> Daredevil            | Specialist      | Marvel    | 38.54%
-> Hawkeye              | Specialist      | Marvel    | 38.54%
-----

ESITO SIMULAZIONE: Task-force a basso profilo identificata tramite minimizzazione d'angolo.
Parametri di mobilità e anonimato geometricamente soddisfatti.
```

denominato "**MISSIONE SEGRETA (Infiltrazione spionistica)**".
 Il sistema formula una query semantica basata sul target `'is_classified_as_Specialist has_trait_low_profile has_trait_high_mobility'`.
 Attraverso il calcolo della similarità del coseno, il *Decision Support System* identifica la task-force ottimale estraendo i quattro eroi con il punteggio di affinità geometrica più elevato.
 Tutti i membri suggeriti appartengono correttamente alla classe **Specialist**, dimostrando l'efficacia del meccanismo di *matching* nel soddisfare i parametri logici di mobilità e anonimato richiesti dal contesto tattico.

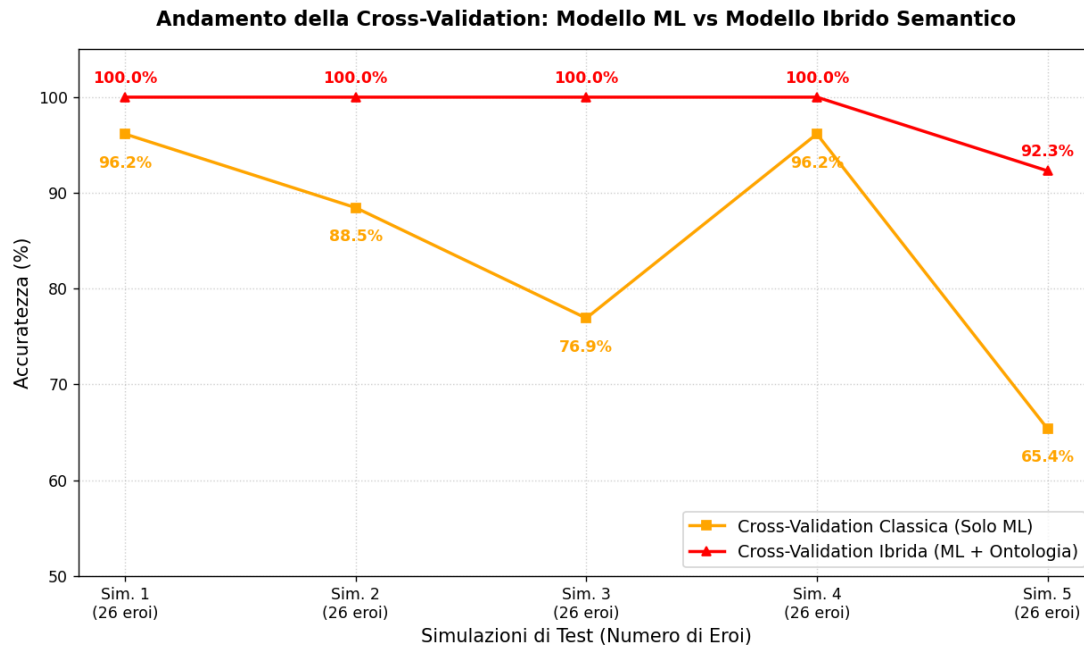
8 Conclusioni e Sviluppi futuri

'*HEROES IN SUPER-TRAINING*' ha dimostrato con successo l'efficacia e la robustezza dell'integrazione tra tecniche di Machine Learning statistico e metodologie di modellazione semantica basate su ontologie OWL, applicate alla strutturazione di un *Decision Support System* (DSS) nel dominio dei supereroi.

Il **Machine Learning**, pur mostrando un'elevata capacità di apprendimento dai dati storici grezzi, soffriva di una marcata instabilità nei casi di test più complessi, registrando crolli verticali nell'accuratezza (fino al 65.4%).

L'**Infrastruttura Semantica** basata sulla TBox OWL e sul ragionatore HermiT ha introdotto un livello di determinismo logico essenziale, agendo come un vincolo algoritmico a protezione delle anomalie statistiche.

I risultati della Cross-Validation a 5 fold hanno confermato la validità di questa architettura ibrida. Come abbiamo notato precedentemente con il seguente grafico.



L'ontologia ha dunque dimostrato il suo valore non come classificatore isolato, ma come fondamentale supporto cooperativo per il modello di Machine Learning. Lo sviluppo del modulo `decision_system()` ha permesso di tradurre la conoscenza estratta in uno strumento operativo concreto.

Sviluppi futuri?

Un promettente sviluppo futuro per il sistema ibrido consiste nell'integrazione di **Datalog** (o di linguaggi di regole basati su di esso, come SWRL - *Semantic Web Rule Language*).

L'introduzione di un motore di ragionamento basato su Datalog permetterebbe di modellare dinamiche di combattimento più avanzate, di formalizzare le euristiche spostandole direttamente all'interno dello strato logico formale della Base di Conoscenza.

In sintesi si eleverebbe la complessità e il realismo delle simulazioni all'interno della fase di *Battle*.

9 Bibliografia

Riferimenti bibliografici

- [1] Poole, D., & Mackworth, A. *Supervised Machine Learning*.
In: Artificial Intelligence: Foundation of Computational Agents, Capitolo 7.
- [2] Poole, D., & Mackworth, A. *Knowledge Graph and Ontologies*.
In: Artificial Intelligence: Foundation of Computational Agents, Capitolo 16.
- [3] Oracle Corporation. *MySQL Database Service*.
<https://www.mysql.com/it/>
- [4] Pandas. *Getting Started*.
https://pandas.pydata.org/docs/getting_started/index.html#getting-started
- [5] Pandas. *User Guide*.
https://pandas.pydata.org/docs/user_guide/index.html
- [6] Stack Overflow. *Convert text columns into numbers in sklearn*.
<https://stackoverflow.com/questions/34915813/convert-text-columns-into-numbers-in-sklearn>
- [7] Scikit-learn. *DecisionTreeClassifier*.
<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
- [8] Scikit-learn. *LabelEncoder*.
<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html>
- [9] Matplotlib. *Quick Start*.
https://matplotlib.org/stable/users/explain/quick_start.html
- [10] Stanford Center for Biomedical Informatics Research. *Protégé Ontology Editor*.
<https://protege.stanford.edu/software/>
- [11] Lamy, J. B. (2017). *Owlready2: A Python module for ontology-oriented programming*.
Artificial Intelligence in Medicine, 80, 11-28.
<https://pypi.org/project/owlready2/>
- [12] Scikit-learn. *TfidfVectorizer – Text Feature Extraction*.
https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- [13] Scikit-learn. *Pairwise metrics and distances: cosine_similarity*.
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html